CLAUDE CODE PLUGIN · V0.28.1

AgenTeX User Guide

Agentic test execution for browser, mobile, API, database, Jira and Azure DevOps — step by step, with real examples.

Playwright | Appium | API & Swagger | SQL Server | Jira Cloud | Azure DevOps | Figma | CI/CD

Source: github.com/MhmdElGazzar/agentex · Guide v2 for AgenTeX v0.28.1 (commit a6bb627) · November 2026

Table of Contents

Click any title or page number to jump to it. Every chapter has a "Back to contents" link, and the PDF bookmarks panel shows the same structure.

1. What is AgenTeX? . . . 5

1.1 What it does for a tester . . . 5

1.2 Safety guarantees — and what enforces each one . . . 6

1.3 The two run modes . . . 8

1.4 All commands at a glance . . . 8

1.5 How it fits together . . . 9

1.6 When to use AgenTeX . . . 9

2. Quick start by goal — "I want to..." . . . 10

3. Installation and first setup . . . 12

3.1 Prerequisites . . . 12

3.2 Install the plugin . . . 12

3.3 Install the browser driver . . . 12

3.4 Scaffold the project with /init-test . . . 13

3.5 Set permissions (important) . . . 13

3.6 Your first 15 minutes . . . 14

3.7 New to Claude Code? The basics . . . 14

3.8 Running bundled scripts yourself — the plugin path . . . 15

4. Configuration . . . 16

4.1 The golden rule: envSecret . . . 16

4.2 config/project.json . . . 16

4.3 environments/<env>.json . . . 17

4.4 .env — secrets only . . . 19

5. Writing test specs . . . 20

5.1 Anatomy of a spec . . . 20

5.2 Step types you can mix in one spec . . . 21

5.3 Stateful chains — keep dependent steps in one file . . . 21

5.4 Organising specs into suites . . . 21

5.5 Result statuses you will see . . . 22

6. Browser testing . . . 23

6.1 Sequential run (human-in-the-loop) . . . 23

6.2 Run a suite folder . . . 23

6.3 Parallel regression (autonomous) . . . 23

6.4 What you get . . . 24

6.5 Tips for good browser runs . . . 24

7. Define flow — build a spec by doing it . . . 25

7.1 Defining a new flow . . . 25

7.2 Clarifying an existing spec . . . 25

8. Optimize login — log in once, reuse it . . . 27

8.1 Turn it on . . . 27

8.2 Captcha and OTP . . . 27

8.3 Check a saved session manually . . . 27

9. Mobile testing (Android / iOS) . . . 29

9.1 Step 1 — Install Appium and a driver . . . 29

9.2 Step 2 — Add a mobile block to your environment . . . 29

9.3 Step 3 — Write (or use) a mobile spec . . . 30

9.4 Step 4 — Run it . . . 31

9.5 How Claude talks to the device . . . 32

9.6 Mobile permissions . . . 32

9.7 Output . . . 32

10. API testing . . . 33

10.1 Step 1 — Define requests in the catalog . . . 33

10.2 Step 2 — Use api: steps in a spec . . . 34

10.3 Standalone API pass — /test-endpoints . . . 34

10.4 The suite editor — /test-endpoints --edit . . . 35

10.5 Generate tests from Swagger / OpenAPI . . . 36

11. Database checks . . . 37

11.1 Step 1 — Define queries . . . 37

11.2 Step 2 — Install sqlcmd . . . 37

11.3 Step 3 — Use it in a spec . . . 37

11.4 Safety rules (enforced in code) . . . 38

12. UI check — compare with Figma or a screenshot . . . 39

12.1 Syntax . . . 39

12.2 Setup . . . 39

12.3 Modes . . . 39

12.4 Viewports and results . . . 40

13. Ask the knowledge base . . . 41

13.1 Setup . . . 41

13.2 Use it . . . 41

14. Jira Cloud . . . 42

14.1 One-time setup . . . 42

14.2 What your Jira project needs . . . 43
14.3 Estimate QA effort . . . 43
14.4 Design test artifacts — /design-test . . . 43
14.5 File bugs after a run . . . 43
14.6 Known limitations on Jira . . . 44
14.7 Jira troubleshooting . . . 44

15. Azure DevOps . . . 45

15.1 One-time setup . . . 45
15.2 Estimate a sprint . . . 45
15.3 Design test cases . . . 46
15.4 Approval and the ledger . . . 47
15.5 File bugs after a run . . . 47
15.6 Reach Azure resources mid-run . . . 47

16. Reports and evidence . . . 48

16.1 What a reported defect looks like . . . 48
16.2 The dashboard . . . 48

17. CI quality gate . . . 50

17.1 Exit codes — the contract . . . 50
17.2 The command . . . 50
17.3 Runner prerequisites . . . 51
17.4 Example — GitHub Actions . . . 51
17.5 CI mode rules . . . 52

18. Git and security . . . 53

18.1 What to commit . . . 53
18.2 Environment safety . . . 53
18.3 Secret scan and pre-commit hook . . . 53

19. Keeping your project up to date . . . 55

20. Troubleshooting and FAQ . . . 56

20.1 Common problems . . . 56
20.2 Supported, not supported, experimental (v0.28.1) . . . 57
20.3 Frequently asked questions . . . 57

B. Appendix — Glossary . . . 58

C. Appendix — Cheat sheet . . . 59

1 What is AgenTeX?

↑ Back to contents

AgenTeX (**Agen**tic **Te**st e**X**ecution) is a QA plugin for Claude Code. You tell it what to test, in plain English or with a short spec file. An agent then plans the scenarios, drives a **real browser** (Playwright CLI) or a **real mobile app** (Appium), calls your APIs and database, takes screenshots and logs as evidence, and writes a full defect report.

> **VERSION**
>
> This guide matches **AgenTeX v0.28.1** (repo commit `a6bb627`, 26 Oct 2026). Check your version with `/plugin` or in `.agentex/version.json`. After a plugin update, check the repo's `CHANGELOG.md` for changes this guide doesn't cover yet.

1.1 What it does for a tester

Area	What AgenTeX does
Web / browser testing	Runs your scenarios in a real Chromium browser. Checks visible results, console errors and failed network calls.
Mobile testing	Runs the same kind of flow on native Android / iOS apps through Appium, and watches the device log for crashes.
API testing	Runs `api:` steps inside specs, or a full standalone API pass with `/test-endpoints`. Can generate the tests from Swagger / OpenAPI.
Database checks	Runs `db:` steps: only the named SQL queries you defined, never its own SQL. DDL is always refused.
Design checks	`ui-check:` steps compare the live page with a Figma frame or a baseline screenshot.
Jira / Azure DevOps	Estimates QA effort and creates `[Testing]` tasks, designs test cases from acceptance criteria, files bugs with screenshots. Always behind **one** approval screen.
Reports	Every run writes `report.md`, an interactive `extent-report.html` dashboard, screenshots, logs and a merged bug list.
CI/CD	A headless quality gate for GitHub Actions or Azure Pipelines with clear exit codes: 0 PASS, 1 FAIL, 2 BLOCKED.

1.2 Safety guarantees — and what enforces each one

Where AgenTeX writes files

Activity	Writes to
Test runs (`/execute-test` , mobile, API pass)	`executions/` only, plus `test/.auth/` when saving a login session
`/define-flow`	New spec files under `test/`
`/init-test` , `/update-agentex`	`config/` , `environments/` , `integration/` , `test/` , `.env` , `.agentex/` , `.gitignore` , `CLAUDE.md`
`/import-swagger` , suite editor	`integration/` , `.agentex/cache/` , `test/.auth/api-tokens/`
Estimation, test design	`config/estimation.json` , `.agentex/` , and your tracker (after approval)
Your application source	**Never** by design. Enforced only by the `Edit(...)` deny rule, see below.

Not every guarantee is enforced the same way. Some are checked in code and cannot be bypassed; others depend on the permission rules you copy (3.5) or on the agent following its instructions. Know which is which.

Guarantee	Enforced by	What you must do
`db:` steps run only catalogued queries; DDL (`DROP/TRUNCATE/ALTER`) refused; parameters sanitised	**Code** (`run_db.js`)	Nothing. A bare `sqlcmd` outside the runner still prompts you.
`api:` steps run only catalogued requests	**Code** (`run_api.js`)	Nothing. A bare `curl` prompts you.
Tracker writes (Jira / Azure) only after one approval; none at all in CI	**Code** (bundled scripts, `AGENTEX_CI=1` refusal)	Read the approval screen before approving.
Mobile app data not reset without consent	**Code** (`appium_client.js`) + agent rules for raw REST	Approve a reset only when you want one.
Claude cannot read `.env` , keys or saved sessions	**Your permissions** (deny rules)	Copy the `deny` block into `.claude/settings.json` . Without it, this is not enforced.
Your application source is never edited	**Your permissions** (`Edit(./src/**)`) + agent rules	Point the rule at your real source folder. If your code is not in `./src` , the default rule protects nothing.
Secret values are never printed or put on a command line	**Code** for the bundled scripts (`fill_secret.js` , `--text-env` , header-only tokens) + **agent rules** for page echoes, console logs, request bodies and traces	Declare secrets as `envSecret` . Review screenshots before sharing them; they are not scanned.
No captcha / OTP bypass. In **UI runs**, no account creation, payments or real personal data	**Agent rules**	Use test environments and disposable test users.

Not guaranteed — these really run: catalogued `api:` writes (POST / PUT / DELETE: *the catalog is the authorisation*), and every step you approve in `/define-flow` , including add / edit / delete. There is **no code check that blocks production**. A DELETE or payment endpoint in the catalog will run against whatever `baseUrl` the environment points to.

> **BOTTOM LINE**
>
> "Code" guarantees hold even if the agent makes a mistake. "Your permissions" guarantees hold only after you set up 3.5 correctly. "Agent rules" guarantees are strong defaults, not hard walls, so keep AgenTeX on non-production environments.

1.3 The two run modes

Mode	How to start it	Behaviour
Sequential (default)	`/execute-test <url>` or a plain-English request	Human-in-the-loop. Claude proposes scenarios, waits for your OK, runs one at a time and pauses after each one. Best for exploratory and first runs.
Parallel (autonomous)	"Run a **parallel** regression ... from the specs in `test/suite1/`"	One isolated sub-agent and browser session **per spec file**, no pauses. Results are merged into one report. Best for regression suites.

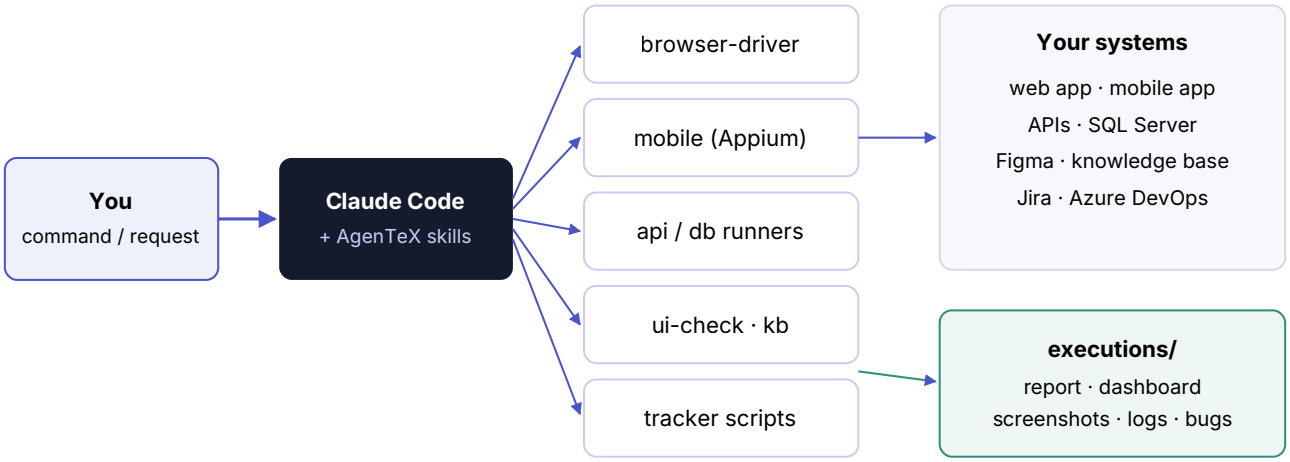
1.4 All commands at a glance

Command	What it does	Chapter
`/init-test`	Sets up AgenTeX in your project (sample specs, config, environment, `.env`, tracker choice).	3
`/execute-test <url | suite>`	Runs browser / API / DB specs.	6
`/define-flow <url | spec>`	Builds a spec live, step by step, while it executes.	7
`/execute-mobile-test <suite>`	Runs native Android / iOS specs through Appium.	9
`/test-endpoints [--edit]`	Runs every catalogued API test case; `--edit` opens the suite editor.	10
`/import-swagger <file|url> [--wizard]`	Generates the API catalog and test suite from Swagger / OpenAPI JSON.	10
`/ask-kb <question>`	Asks your project knowledge base (advisory only).	13
`/estimate-story <ids>`	Estimates QA effort for named stories and creates `[Testing]` tasks.	14, 15
`/estimate-sprint`	The same for every story in the current sprint.	14, 15
`/design-test <ids>`	Turns story acceptance criteria into linked test cases.	14, 15
`/update-agentex`	After a plugin update: migrates your project to the new conventions, keeping your values.	19

> **SLASH COMMANDS ARE OPTIONAL**
>
> You can always just describe what you want in plain English, for example: *"Test https://example.com — the signup form: happy path plus empty and bad-email cases."* Claude picks the right skill by itself.

1.5 How it fits together

You type a command or a request. Claude picks the matching **skill**, which sends each step to a **driver**. In parallel runs, work goes to **sub-agents**. Every result lands in `executions/`.



In parallel mode, Claude starts one sub-agent per spec file (`qa-executor`, `mobile-qa-executor`, `api-executor`), each with its own session, and merges their results.

1.6 When to use AgenTeX

- Manual regression you repeat every sprint (6.3, 9.4).
- Exploratory first runs where you want to watch and steer (6.1).
- Flows that cross UI + API + database (10.2, 11.3).
- Turning acceptance criteria into test cases and estimates (14, 15).
- Writing a reliable spec by doing it (7).
- A quality gate in your pipeline (17).

2 Quick start by goal — "I want to..."

↑ Back to contents

Find your goal below, run the command, then click the chapter link for the full step-by-step guide.

Test a website in a browser

```
/execute-test https://your-app.com
```

Claude plans scenarios, you approve, it drives Chromium.

→ Chapter 6 — Browser testing

Test an Android / iOS app

```
/execute-mobile-test suite1
```

Add a `mobile` block to your environment file, start Appium, run.

→ Chapter 9 — Mobile testing

Test APIs (no browser)

```
/test-endpoints
```

Runs every case in your API suites and reports pass / fail per case.

→ Chapter 10 — API testing

Generate API tests from Swagger

```
/import-swagger --wizard ./openapi.json
```

A guided page that finds real test data and previews 4 checks per endpoint.

→ 10.5 — Swagger import

Check the database after a UI action

```
db: sample-db.todo-by-title(title=qa-test-item)
```

Add a `db:` line to a spec; only catalogued queries run.

→ Chapter 11 — Database checks

Write a spec without writing it

```
/define-flow https://your-app.com on uat
```

Claude proposes each step, runs it live, you confirm.

→ Chapter 7 — Define flow

Work with Jira

```
/estimate-sprint · /design-test PROJ-12
```

Sub-tasks with time tracking, test artifacts, bug filing.

→ Chapter 14 — Jira

Work with Azure DevOps

```
/estimate-story 12345 · /design-test 12345
```

[Testing] tasks, Test Cases linked "Tested By", Bugs.

→ Chapter 15 — Azure DevOps

Compare the page with Figma

```
ui-check: figma 123:456 — mode: exact
```

A spec step that compares the live page with a design.

→ Chapter 12 — UI check

Stop logging in every test

```
"login": { "mode": "session" }
```

Log in once, save the session, reuse it (~8 s instead of ~197 s).

→ Chapter 8 — Optimize login

Run tests in CI/CD

```
node …/ci_gate.js --suite test/suite1/ --env uat
```

Headless gate with exit codes 0 / 1 / 2.

→ Chapter 17 — CI quality gate

File the bugs a run found

```
"File the defects from the last run as bugs"
```

Validated fields, annotated screenshots, one approval.

→ 14.5 · 15.5

Check Azure resources mid-test

```
"Check if the latest deployment succeeded"
```

Read-only `az` CLI: App Service logs, Storage, Key Vault, AKS.

→ 15.6 — Azure resources

Ask the project knowledge base

```
/ask-kb how does checkout work?
```

Advisory context, never pass/fail evidence.

→ Chapter 13 — Ask the KB

3 Installation and first setup

↑ Back to contents

Do this once per machine (steps 1–2) and once per project (steps 3–5). It takes about 10 minutes.

3.1 Prerequisites

What	Why / when
Claude Code	AgenTeX is a Claude Code plugin. New to it? See 3.7.
Node.js 20+ and npm	All bundled scripts are Node scripts. Needed always.
@playwright/cli	Only for browser specs. Installed in step 2.
Appium + driver	Only for mobile testing (9.1).
sqlcmd	Only for `db:` steps on SQL Server (11.2).
az CLI	Only to read Azure resources mid-run. Not needed for Azure DevOps boards.

3.2 Install the plugin

1. **Open Claude Code** in the folder of the project you want to test.
2. **Add the marketplace** and **install** AgenTeX:

```
/plugin marketplace add MhmdElGazzar/elgazzar-plugins
/plugin install agentex@elgazzar-plugins
```

3. **Restart / reload** Claude Code if asked, then type `/`. You should see `/init-test`, `/execute-test` and the other commands.

3.3 Install the browser driver

In your project folder (Claude also offers to do this for you):

```
npm install -D @playwright/cli
npx playwright-cli install-browser chromium
```

> **TIP**
> Specs with no browser steps do not need a browser. They declare `Drivers: api`, `Drivers: db` or `Drivers: api, db` in the header. You can skip this step if you only test APIs.

3.4 Scaffold the project with `/init-test`

1. **Run** `/init-test`.
2. **Answer the wizard.** It asks which work tracker you use: **Azure DevOps**, **Jira Cloud** or **none**. Only that tracker gets configured.
3. **Review what was created:**

```
your-project/
├── config/project.json        # project settings (tracker, login mode, KB, Figma)
├── environments/qc.json       # target URL, test users, db / api blocks
├── .env                       # SECRETS ONLY — gitignored
├── integration/               # API + DB catalog (the only calls tests may make)
│   ├── sample_api.json
│   └── sample_db.json
├── test/
│   └── suite1/                # sample specs — edit them for your app
│       ├── product-search.md
│       └── signup-form.md
├── executions/                # run results land here
├── .agentex/version.json      # plugin version this project matches
└── CLAUDE.md                  # project guidance for Claude
```

The wizard can also install an optional **pre-commit hook** that scans for secrets (see Chapter 18).

3.5 Set permissions (important)

Plugins cannot ship permission rules, so copy them yourself. Without this step, Claude stops to ask before every single browser command.

1. **Open** `settings.example.json` from the AgenTeX repo.
2. **Copy** its `"permissions"` block into your project's `.claude/settings.json`. Merge it with anything already there.
3. **Fix two entries** for your project:
   - `Edit(./src/**)` protects your app source only if it really lives in `./src`. Point it at your real source folder.
   - **Recommended:** replace `Bash(node:*)` with `Bash(node <installed plugin path>/skills:*)`, which allows only AgenTeX's own scripts. Run `/plugin` to see the path. Keep the broad `Bash(node:*)` only on a personal sandbox: it allows *any* node command.

A shortened view of what the block does:

```
"permissions": {
  "allow": [ "Bash(npx playwright-cli:*)", "Bash(node:*)", "Bash(adb devices:*)",
             "Bash(npx appium server:*)", "Bash(az account:*)", ... ],
  "ask":   [ "Bash(curl:*)", "Bash(sqlcmd:*)", "Bash(adb uninstall:*)",
             "Bash(npx appium driver install:*)", ... ],
  "deny":  [ "Read(./.env)", "Read(./**/*.pem)", "Read(./test/.auth/**)",
             "Edit(./src/**)", "Bash(rm -rf:*)", "Bash(git push:*)", ... ]
}
```

> **HOW RULES ARE CHECKED**
>
> **deny** first, then **ask**, then **allow**. So an *ask* entry can carve a prompt out of a broader *allow*. Example: `close-all` still prompts even though `npx playwright-cli:*` is allowed.

3.6 Your first 15 minutes

Minute	Do
0–3	Install the plugin (3.2) and the browser driver (3.3).
3–6	`/init-test`, choose your tracker or "none" (3.4).
6–9	Copy the permissions block (3.5) and fix the `Edit(...)` path.
9–10	Set `portalUrl` in `environments/qc.json` to a **test** environment of your app.
10–15	Run the steps below and open the report.

1. **Edit the target.** Open `environments/qc.json` and set `"portalUrl"` to your app, or edit `Target:` in `test/suite1/signup-form.md`.
2. **Run** `/execute-test https://your-app.com`
3. **Approve the plan.** Claude restates what it will test and lists numbered scenarios. Nothing runs until you approve.
4. **Watch and confirm.** After each scenario Claude reports PASS / FAIL with a screenshot and pauses.
5. **Open the results** in `executions/execu_<timestamp>/`: `report.md`, `extent-report.html` and `bugs/bug-list.md`.

3.7 New to Claude Code? The basics

- **Type a request** in plain English, the way you would ask a colleague.
- **Slash commands** (`/execute-test …`) are shortcuts that run one exact ability.
- **Approving actions:** Claude pauses before running a new command or before an AgenTeX action such as filing a bug. Nothing happens until you respond. This is a safety feature.

- **AgenTeX checkpoints** come on top of that: a sequential run stops after planning and after every scenario.

3.8 Running bundled scripts yourself — the plugin path

Some sections show commands such as `node ${CLAUDE_PLUGIN_ROOT}/skills/…`. `${CLAUDE_PLUGIN_ROOT}` only exists **inside Claude Code**. In your own terminal it is empty, and the command fails.

- **Easiest:** ask Claude to run it, e.g. *"check my saved session for qc / valid_user"*.
- **By hand:** run `/plugin` in Claude Code to see the installed path, and use it in place of `${CLAUDE_PLUGIN_ROOT}` (written as `<plugin-root>` in this guide).
- **In CI:** use a pinned checkout of the repo as `<plugin-root>`. Never hardcode `~/.claude/plugins/…`; that cache changes across updates.

> **OTHER AI TOOLS**
>
> AgenTeX is also published for **OpenAI Codex**, **GitHub Copilot** and **Devin** as separate branches of the repo (see the `hosts/` folder). This guide uses Claude Code.

4 Configuration

↑ Back to contents

Every kind of project data has exactly one home. Learn these three files and you know the whole setup.

Kind	Examples	Home
Secrets	PAT, API tokens, passwords	`.env` (only these)
Project settings	Tracker, login mode, KB, Figma, CI policy	`config/project.json`
Environment data	Portal URL, DB, API, test users, default OTP	`environments/<env>.json`

4.1 The golden rule: `envSecret`

JSON files never hold a real secret. A secret field points to a variable in `.env` by name:

```
"password": { "envSecret": "DEFAULT_PASSWORD" }      // in environments/qc.json
DEFAULT_PASSWORD=MyQaPass123                          # in .env
```

A plain string is accepted only for shared, throw-away test credentials, and the secret scan warns on it.

4.2 `config/project.json`

The real template the wizard creates (shortened):

config/project.json

```
{
  "name": "my-project",
  "defaultEnvironment": "qc",
  "kb":    { "baseUrl": "", "project": "" },
  "figma": { "fileKey": "", "token": { "envSecret": "FIGMA_TOKEN" } },
  "login": { "mode": "session" }
}
```

Key	Purpose
`defaultEnvironment`	Environment used when a run doesn't name one.
`azure` **or** `jira`	Tracker settings. A project has **one** tracker block, never both (14.1, 15.1).
`login.mode`	`"session"` reuses a saved login; `"fresh"` logs in through the UI every run (Ch. 8).
`kb`	Knowledge-base settings (Ch. 13).
`figma`, `viewports`	Design source and viewport sizes for `ui-check:` (Ch. 12).
`ci`	Optional CI gate policy (Ch. 17).

4.3 `environments/<env>.json`

One file per environment: `qc.json`, `uat.json`, `live.json`... This is the real sample from the repo:

environments/qc.json

```
{
  "portalUrl": "https://example.com",
  "defaults": {
    "otp": "0000",
    "password": { "envSecret": "DEFAULT_PASSWORD" }
  },
  "users": {
    "valid_user":   { "phone": "0550000001", "role": "customer" },
    "expired_user": { "phone": "0550000002", "notes": "for negative login scenarios" }
  },
  "db": {
    "server": "localhost", "port": 1433, "name": "my-database",
    "user": "qa_user", "password": { "envSecret": "SQLCMDPASSWORD" }
  },
  "api": {
    "baseUrl": "https://jsonplaceholder.typicode.com",
    "token": { "envSecret": "API_TOKEN" }
  }
}
```

Key	Purpose
`portalUrl`	The app under test (required for browser specs).
`defaults`	Static test values: `otp`, `captcha`, shared `password`.
`users`	Test accounts by handle. A spec says "login as `expired_user`". A user without a password uses `defaults.password`.
`users.<name>.disabled`	A reason string, e.g. `"account locked"`. The suite wizard never signs in as that user.
`db` / `api`	Connection for `db:` / `api:` steps.
`mobile`	Appium target (opt-in, 9.2).
`nonProduction: true`	Allows the suite wizard to create missing test data through the UI, after your consent.

> **EXAMPLE — CHOOSE THE ENVIRONMENT AT RUN TIME**
>
> Say *"run on uat"*, or put `env: uat` in a spec. AgenTeX then reads `environments/uat.json`. Without that, it uses `defaultEnvironment`. Naming an environment with no file is an error, never a silent fallback.

Which base URL and token does an `api:` step use?

Catalog setting	Base URL and auth come from
`"target": "environment"`	Always the active environment's `api.baseUrl` and `api.token`.
`"target": "catalog"` (every Swagger import)	The catalog's own `baseUrl` and `auth`; the environment token is never sent.
No `target`, `baseUrl` empty or `"${API_BASE_URL}"`	The environment (like the sample catalog).
No `target`, any other `baseUrl`	The catalog's own `baseUrl` and `auth`.

For a token, the variable named by the catalog's `tokenEnv` in your shell or `.env` always wins over a token captured by the suite wizard. Legacy `API_BASE_URL` / `API_TOKEN` in `.env` are a fallback when the environment has no `api` block.

Adding a second environment (e.g. UAT)

1. **Copy** `environments/qc.json` to `environments/uat.json`.
2. **Change** `portalUrl`, users and the `db` / `api` blocks for UAT.
3. **Use different secret names** if the values differ, e.g. `{ "envSecret": "SQLCMDPASSWORD_UAT" }`, and add `SQLCMDPASSWORD_UAT=` to `.env`.
4. **Run** `/execute-test suite1 on uat`.

4.4 `.env` — secrets only

```
# Azure DevOps
AZURE_PAT=
# Jira Cloud
JIRA_EMAIL=
JIRA_API_TOKEN=
# Database (read natively by sqlcmd)
SQLCMDPASSWORD=
# API bearer token for api: steps
API_TOKEN=
# Others
KB_ASK_API_KEY=
FIGMA_TOKEN=
DEFAULT_PASSWORD=
```

Fill only what you use. Any variable named in an `{ "envSecret": "…" }` field belongs here too, e.g. `SQLCMDPASSWORD_UAT`.

> **NEVER COMMIT .ENV**
>
> `/init-test` adds it to `.gitignore`. With the permission rules from 3.5, Claude is denied reading it; the bundled scripts read it themselves and send values only in request headers.

5 Writing test specs

↑ Back to contents

A spec is a plain markdown file: a target, what "correct" means, and numbered scenarios in plain language. No code.

5.1 Anatomy of a spec

This is the real sample `test/suite1/signup-form.md` from the repo:

test/suite1/signup-form.md

```
# Spec: Signup form validation

Target: https://example.com/signup
Type: form validation — NO real account is created (validation-only)

## Acceptance criteria
- Valid input reaches a visible success/confirmation state.
- Invalid input is rejected with a specific, visible error message; the form must not submit.
- No console errors or failed network calls during any scenario.

## Scenarios
1. **Happy path** — fill Name, a disposable email (`qa.tester@example.com`), and a valid
   password, then submit. Expect a visible success confirmation.
2. **Empty required fields** — submit with every field blank. Expect an inline
   "required" error on each field; the form must not submit.
3. **Bad email format** — enter `not-an-email` and submit. Expect an email-format error.
4. **Weak password** — enter a 3-character password and submit. Expect a strength error.

## Notes
- Screenshot every scenario (pass and fail).
- Treat any console error or failed request as a defect even if the UI looks fine.
```

Part	What to write
`Target:`	The URL or page under test.
`Type:`	A short description, and whether real data is created.
`Drivers:`	Optional: `api`, `db`, `mobile`. Without it, the spec is a browser spec.
Acceptance criteria	What "pass" means. Include "no console errors / failed calls".
Scenarios	Numbered: happy path, edge cases, negative cases.
Notes	Stateful order, disposable data, special rules.

5.2 Step types you can mix in one spec

Step in a spec	Handled by	Example
Plain prose	browser-driver	`Click "Search" and expect results`
`api: …`	api-integration	`api: sample-api.get-todo(id=1) → expect HTTP 200`
`db: …`	db-integration	`db: sample-db.todo-by-title(title=x) → expect 1 row`
`kb: …`	ask-kb (advisory)	`kb: how is the discount applied?`
`ui-check: …`	ui-check (needs browser)	`ui-check: figma 123:456 — mode: exact`

5.3 Stateful chains — keep dependent steps in one file

One spec file = one browser session. If steps depend on each other (login → action → assert), keep them in the same file and mark them stateful. The real sample `product-search.md`:

test/suite1/product-search.md

```
# Spec: Product search
Target: https://example.com
Type: search / results — read-only, stateful chain

## Scenarios  (stateful — run in order, in one session)
1. Search a common term (e.g. `shirt`); expect at least one result rendered.
2. Search a nonsense term (e.g. `zzzxqq`); expect a visible "no results" message.
3. Clear the search box; expect the default/full listing to return.
```

5.4 Organising specs into suites

- A **suite** is just a folder: `test/suite1/`, `test/suite2/`, `test/mobile-suite1/`.
- Keep each file to **one independent feature or flow**. In parallel mode each file gets its own session.
- Run one suite: `/execute-test suite2` → runs only `./test/suite2/`.
- Users: write "login as `valid_user`". The user must exist in the active environment's `users`, or the step is BLOCKED (never improvised).

> **DON'T WANT TO WRITE SPECS BY HAND?**
>
> Use /define-flow. Claude builds the spec while running it live, and you just confirm each step.

5.5 Result statuses you will see

Status	Meaning
PASS	The expected result was seen (checked by real visibility, not just DOM presence).
FAIL	A product defect: wrong behaviour, a console error or a failed network call.
BLOCKED	Could not be judged: a missing user, catalog entry, secret, unreachable target, captcha… The reason is always named.
Warning	Passed with a caveat, e.g. layout drift in a `ui-check` reference mode.
View mismatch	A `ui-check` baseline form factor differs from the run (e.g. mobile design vs desktop run).

6 Browser testing

↑ Back to contents

The core of AgenTeX: describe what to test, and Claude drives a real browser, captures evidence, checks for errors and reports.

6.1 Sequential run (human-in-the-loop)

1. **Start the run** with a command or plain English:

```
/execute-test https://example.com

# or
Test https://example.com — the signup form: happy path plus empty and bad-email cases.
```

2. **Plan.** Claude restates what it understood and lists numbered scenarios (happy, edge, negative). **Nothing runs yet.** Approve, edit or remove scenarios.
3. **Drive.** A real Chromium browser opens. Each scenario runs one at a time with a screenshot (pass or fail) while console errors and failed network calls are watched.
4. **Checkpoint.** After each scenario you see PASS / FAIL with evidence. Claude pauses so you can stop or redirect.
5. **Report.** Everything is written to `executions/execu_<timestamp>/`.

6.2 Run a suite folder

```
/execute-test suite1                    # runs ./test/suite1/
/execute-test test/suite3/ on uat       # a suite against environments/uat.json
/execute-test https://example.com       # target URL, scenarios planned with you
```

A suite folder that doesn't exist yet is created and seeded with a starter spec for you to adapt. With no suite named, `./test/suite1/` is used.

6.3 Parallel regression (autonomous)

1. **Ask explicitly for "parallel":**

```
Run a parallel regression against https://example.com from the specs in test/suite1/.
```

2. **Claude spins up** one `qa-executor` sub-agent with its own browser session per spec file. No approval pauses.

3 **Results merge** into one `report.md`, one dashboard and one merged `bug-list.md`.

> **ISOLATION**
> Each run gets uniquely named browser sessions (never playwright's shared `default`). Two runs, even in two Claude Code windows, never share a browser, and each run closes only the browsers it opened.

6.4 What you get

```
executions/execu_2026-11-10_14-02-11/
├── report.md                          # summary: what passed, what failed
├── extent-report.html                 # interactive dashboard
├── browser-sessions/<session>/
│   ├── logs/                          # console / network captures, api/db evidence
│   └── screenshots/                   # one per scenario
└── bugs/
    ├── bug-list.md                    # merged defect list
    └── screenshots/
```

6.5 Tips for good browser runs

- Write **expected results** that are visible: "a visible success message", not "the request succeeds".
- Use **disposable data** (`qa.tester@example.com`). AgenTeX never creates real accounts or completes payments.
- A **captcha or OTP** is never bypassed. A fixed test OTP in `defaults.otp` is typed; anything else waits for you.
- Logging in every scenario is slow. Turn on Optimize login.

7 Define flow — build a spec by doing it

↑ Back to contents

Instead of writing 15 steps from memory and discovering step 7 was wrong, `/define-flow` builds the spec **live**. Each step is proposed, executed in the browser and confirmed by you before the next one.

7.1 Defining a new flow

1. **Start:** `/define-flow https://your-app.example on uat`
2. **Setup.** Claude resolves the target and environment, logs in if needed, and asks for the flow's goal in one sentence. That becomes the spec title.
3. **One step at a time.** Claude looks at the live page and proposes the next step, e.g. *"I can see a Submit order button — is submitting next?"*, or offers the visible options as choices.
4. **You assert.** The step runs immediately; you see a screenshot and what was observed. Confirm it, or correct it and Claude re-runs it. Corrections are **forward-only**: earlier confirmed steps are locked.
5. **Values flow forward.** When a step produces something (e.g. an order number), Claude offers "use this later?". It is saved symbolically: *"the order number produced in step 3"*. Unique inputs become *"a fresh disposable email"*, so re-runs still pass.
6. **Save.** Say the flow is complete. Claude saves a normal spec, by default under `test/suite1/<slug>.md`.
7. **Prove it twice (optional).** Claude offers to re-run the new spec with `/execute-test`.

> **NOTHING IS LOST**
>
> Every confirmed step is appended to a running draft immediately. A crash at step 14 keeps steps 1–13. If you must stop early, save a partial spec, clearly marked incomplete.

7.2 Clarifying an existing spec

```
/define-flow test/suite1/checkout.md
```

Claude walks the spec live, step by step. Any unclear step (ambiguous target, missing expected result) becomes a question to you. The result is saved as a **new** spec file. Your original only gets one note at the top: "Defined flow available at <path>".

USE A TEST ENVIRONMENT

Because you direct every step, Claude executes what you approve, including add / edit / delete actions. Run define sessions only on an environment you are responsible for.

8 Optimize login — log in once, reuse it

↑ Back to contents

Login is usually the slowest part of a web test. With session mode, Claude pays it once: about **8 seconds** to resume instead of about **197 seconds** to log in, measured on a real project.

8.1 Turn it on

1. **Set the mode** in `config/project.json`:

```
"login": { "mode": "session" }      // "fresh" = log in through the UI every run
```

2. **Run a test as usual.** The first run logs in for real and verifies it by a landmark only a logged-in page shows (never by URL). The session is saved to `test/.auth/<env>-<user>-state.json`.

3. **Later runs** reload that session into a fresh browser and check it is still valid. If it expired, Claude simply logs in again and saves a new one.

8.2 Captcha and OTP

Claude never bypasses them. It runs with a visible browser window and waits for you to complete that one step. The session is then saved, so you do it once.

8.3 Check a saved session manually

```
node "<plugin-root>"/skills/optimize-login/scripts/session.js resume \
  --state  test/.auth/qc-valid_user-state.json \
  --url    https://app.example.com/dashboard \
  --absent "role=button[name='Login']"
```

Needs `npm i -D playwright` and `npx playwright install chromium` in your project. Add `--headed` to watch it, or `--channel chrome` for real Chrome.

Situation	Behaviour
Cookie / localStorage login	Saved and resumed normally.
IndexedDB login	Cannot be resumed; the post-load check fails plainly. Use `"fresh"`.
One file per environment + user	`qc-valid_user` and `uat-valid_user` are separate sessions.
CI	Never saves a login; it may load one the pipeline provides.
Session file from an older version (per-app name)	Ignored; the first run logs in once and saves under the new name.

A SESSION FILE IS A PASSWORD

`test/.auth/` is gitignored. Never commit or publish it. Apps that keep login in IndexedDB cannot be resumed this way: the post-load check fails plainly.

9 Mobile testing (Android / iOS)

↑ Back to contents

The same flow as browser testing, for native apps. Claude drives a real Appium session, takes screenshots, watches the device log (`adb logcat` / simulator log) for crashes, and reports.

9.1 Step 1 — Install Appium and a driver

```
npm install -D appium
npx appium driver install uiautomator2     # Android
npx appium driver install xcuitest         # iOS (macOS + Xcode only)

# Optional: the bundled client wrapper (simpler than raw curl)
npm install -D webdriverio
```

Check the install without triggering a silent download (plain `npx appium` downloads Appium if the project lacks it):

```
npx --no-install appium --version
npx appium driver list --installed
```

Then make sure a device is ready:

```
adb devices                # Android: your emulator/phone must be listed, e.g. emulator-5554
xcrun simctl list          # iOS simulators
npx appium server          # start the server (default http://127.0.0.1:4723)
```

	Android	iOS
Host machine	Windows, macOS or Linux	**macOS + Xcode only**
Appium driver	`uiautomator2`	`xcuitest`
Device	Emulator (Android Studio AVD) or phone with USB debugging; check `adb devices`	Simulator (`xcrun simctl list`) or a provisioned real device
App	`.apk` , or `appPackage` + `appActivity`	`.ipa` / `.app` , or `bundleId`
Device log	`adb logcat`	Simulator log

9.2 Step 2 — Add a `mobile` block to your environment

The block is **opt-in**: the sample `qc.json` has none. Add it to `environments/qc.json` :

environments/qc.json → "mobile"

```
"mobile": {
  "platformName":   "Android",
  "automationName": "UiAutomator2",
  "app":            "/absolute/path/to/app.apk",
  "deviceName":     "emulator-5554"
}
```

Key	Value
`platformName`	`Android` or `iOS`
`automationName`	`UiAutomator2` (Android) / `XCUITest` (iOS)
`app`	Path to `.apk` / `.ipa`, **or** an installed app via `appPackage` + `appActivity` (Android) / `bundleId` (iOS)
`deviceName`, `platformVersion`, `udid`	The target device (`udid` optional)

Claude converts the block to Appium capabilities, like the repo template `sample-android-caps.json`:

```
{ "capabilities": { "alwaysMatch": {
    "platformName": "Android",
    "appium:automationName": "UiAutomator2",
    "appium:app": "/absolute/path/to/app.apk",
    "appium:deviceName": "emulator-5554",
    "appium:noReset": true } } }
```

> **YOUR APP DATA IS PROTECTED**
>
> Claude **always** adds `"appium:noReset": true`. Appium's own default clears app data without asking. A reset (`fullReset`) happens only if you approve it for that run; a parallel run reports BLOCKED instead. Without a `mobile` block, Claude asks you for the app and device instead of guessing.

9.3 Step 3 — Write (or use) a mobile spec

`/execute-mobile-test` copies sample specs into `test/mobile-suite1/` on first run. The real sample `login.md`:

test/mobile-suite1/login.md

```
# Spec: App login
App under test: (via the `mobile` block in environments/<env>.json)
Type: login / validation — no real account is created (validation-only)
Drivers: mobile

## Acceptance criteria
- A valid disposable test user reaches a visible logged-in / home state.
- An invalid password is rejected with a specific, visible error.
- Leaving required fields empty shows an inline "required" error on each field.

## Scenarios
1. **Happy path** — log in as `valid_user`; expect the home screen's landmark element.
2. **Wrong password** — log in as `valid_user` with a wrong password; expect a visible
   "invalid credentials" error, still on the login screen.
3. **Empty fields** — tap login with both fields empty; expect "required" errors.

## Notes
- Treat any app crash, ANR, or stack trace in the device log as a defect.
```

REAL EXAMPLE — FIRST SMOKE RUN WITH NO APP OF YOUR OWN

The repo ships `apidemos-navigation.md`, which targets `io.appium.android.apis` (the official Appium **ApiDemos** sample app). It launches the app, taps "Views", goes back, taps "Graphics", goes back, and fails on any crash or ANR. It is a perfect first test of your Appium setup.

KEEP THE `DRIVERS: MOBILE` LINE

It tells `/execute-test` and the CI gate that this is not a browser spec. They report it BLOCKED ("mobile specs run via /execute-mobile-test") instead of opening a browser.

9.4 Step 4 — Run it

Mode	Command	Behaviour
Sequential	`/execute-mobile-test suite1` or *"Test the login screen of our Android app — happy path plus wrong password and empty fields."*	Plan → approve → one scenario at a time with a pause after each.
Parallel	*"Run a parallel regression against our Android app from the specs in test/mobile-suite1/."*	One `mobile-qa-executor` per spec file, per available device.

`suite1` resolves to `./test/mobile-suite1/`, not `./test/suite1/`.

PARALLEL IS LIMITED BY DEVICES

Each mobile session needs its own physical device, emulator or simulator. Check how many you really have (often 1–2) before asking for a parallel run.

9.5 How Claude talks to the device

Two options; pick what your project has:

- **Raw WebDriver REST** via `curl` to the Appium server. No extra dependency.
- **The bundled wrapper** `appium_client.js` (needs `webdriverio`). Each call prints one JSON line with PASS / FAIL / BLOCKED.

```
node "<plugin-root>"/skills/mobile-testing/scripts/appium_client.js create-session --caps-file caps.json
node .../appium_client.js find      --session <id> --using "accessibility id" --value "Views"
node .../appium_client.js click     --session <id> --element <elementId>
node .../appium_client.js send-keys --session <id> --element <el> --text-env QA_PASSWORD
node .../appium_client.js screenshot --session <id> --out shot.png
node .../appium_client.js back | swipe | source | get-text | close-session ...
```

Locator strategies: `accessibility id` (preferred), `id`, `-android uiautomator`, `-ios predicate string`, `-ios class chain`, `xpath`. Secrets are always typed with `--text-env`, so the value never appears on a command line.

9.6 Mobile permissions

The `settings.example.json` block pre-approves read-only commands (`adb devices`, `adb logcat`, `xcrun simctl list/boot`, `npx appium server`, `driver list`). It **prompts** before `adb uninstall`, `pm clear`, `reboot`, `logcat -c`, `simctl erase/uninstall/spawn` and `appium driver/plugin install`.

9.7 Output

```
executions/execu_<timestamp>/
├── report.md
├── extent-report.html
├── mobile-sessions/<session>/{logs,screenshots}/
└── bugs/{bug-list.md,screenshots/}
```

10 API testing

↑ Back to contents

There are three ways to test APIs: `api:` steps inside a spec, a standalone pass over all API suites, and generating everything from Swagger. They all share one rule: **only catalogued requests run**.

10.1 Step 1 — Define requests in the catalog

The catalog lives in `integration/*_api.json`. This is the real sample `/init-test` creates:

integration/sample_api.json

```
{
  "name": "sample-api",
  "baseUrl": "${API_BASE_URL}",
  "auth": { "type": "bearer", "tokenEnv": "API_TOKEN" },
  "requests": [
    { "name": "get-todo", "method": "GET", "path": "/todos/{id}", "params": ["id"],
      "description": "Fetch a todo by id — verify a record exists / has expected fields" },
    { "name": "create-todo", "method": "POST", "path": "/todos", "params": ["title"],
      "body": { "title": "{title}", "completed": false },
      "description": "Create a todo — example WRITE entry (seeding test data)" }
  ]
}
```

- A request is referenced as `<catalog name>.<request name>`, e.g. `sample-api.get-todo`. Names must be unique.
- `{id}` placeholders are filled from the step's parameters.
- The token comes from `API_TOKEN` in `.env`; the base URL from the environment's `api.baseUrl`.
- Writes (POST / PUT / DELETE) run if catalogued: **the catalog is the authorisation**.

> **PRODUCTION SAFETY — READ THIS BEFORE ADDING WRITE REQUESTS**
>
> Nothing in the runner checks whether `baseUrl` is production. A DELETE, refund or payment request in the catalog runs wherever the active environment points. Keep write requests in catalogs used only with test environments, never put production credentials in `.env` on a test machine, and review catalog changes in code review like any other code.

Request bodies, auth headers and assertions

What	How
Typed body values	`"body": { "qty": { "$param": "qty", "type": "integer" } }`. Types: string, integer, number, boolean, json. A value that doesn't convert is BLOCKED.
Auth header	`auth.type`: `bearer`, `basic`, `apiKey` (custom header: `{"type":"apiKey","headerName":"X-Api-Key","tokenEnv":"API_TOKEN"}`) or `none`. The value is `[redacted]` in logs.
Query params	List them in `"query": ["page"]`, or leave them out of the path and body.
Assertions	Status (`→ expect HTTP 200`), a field exists (`title present`), a field equals a value, a field contains text.

10.2 Step 2 — Use `api:` steps in a spec

Mix them with UI steps, for example to prove a signup really created the user:

```
## Scenarios
1. **Create a user via the UI** — fill the signup form and submit; expect a success state.
2. api: sample-api.get-todo(id=1) → expect HTTP 200 and title present
3. db:  sample-db.todo-by-title(title=qa-test-item) → expect 1 row
```

Or an API-only spec that needs **no browser**:

```
# Spec: Catalog API smoke
Drivers: api

## Scenarios
1. api: shop-api.product-by-sku(sku=PRD-1) → expect HTTP 200 and name present
2. api: shop-api.stock-by-sku(sku=PRD-1) → expect HTTP 200 and quantity present
```

And a DB-only spec, also without a browser:

```
# Spec: Todo data integrity
Drivers: db

## Scenarios
1. db: sample-db.todo-by-title(title=qa-test-item) → expect 1 row
```

Each step writes a PASS / FAIL / BLOCKED JSON result to the session's `logs/`. An entry that is not in the catalog is **BLOCKED**, never guessed.

10.3 Standalone API pass — `/test-endpoints`

Suites (`*_suite.json`) give each catalog entry concrete params and expected results. The real sample:

sample_suite.json

```
{
  "name": "sample-api-suite",
  "cases": [
    { "name": "get-todo-happy-path", "entry": "sample-api.get-todo",
      "params": { "id": 1 },      "expect": { "status": 200, "fields": ["title"] } },
    { "name": "get-todo-not-found",  "entry": "sample-api.get-todo",
      "params": { "id": 999999 }, "expect": { "status": 404 } }
  ]
}
```

1. **Run** `/test-endpoints`. An `api-executor` sub-agent runs every case in every suite.
2. **Read** `executions/execu_<ts>/api-tests/result.md`: a PASS / FAIL / BLOCKED tally plus the defect list, with one evidence log per case.

Optional case fields:

Field	Effect
`"auth": "none"`	Send with no credential, e.g. to test a 401.
`"omit": ["field"]`	Send without that field, e.g. to test a 400.
`"expect": {"fields": [...]}`	Dot-paths that must exist in the JSON response.
`"expect": {"contains": {"message": "token"}}`	Passes when the field includes the text.

EXAMPLE — A "NO TOKEN" CASE

```
{ "name": "get-order-no-token", "entry": "orders.get-order", "auth": "none",
  "params": { "orderId": 1042 },
  "expect": { "status": 401, "contains": { "message": "token" } } }
```

10.4 The suite editor — `/test-endpoints --edit`

- A local page (127.0.0.1 only, per-run access token) lists every suite under `integration/api_test_suites/`.
- Each value has a source badge: UI · Project · Auto · Placeholder · Manual. Filters: Needs review / Failing / Skipped / Blocked.
- Edit a case in the drawer, then **Save**. If the file changed on disk meanwhile, Save refuses and keeps both versions.
- **Run all** or **Run this case** uses the same runner and writes the same evidence to `executions/`.
- Arabic is the default language; add `--lang=en` for English.

10.5 Generate tests from Swagger / OpenAPI

Direct import

```
/import-swagger ./openapi.json --name acme-api
/import-swagger https://petstore3.swagger.io/api/v3/openapi.json
```

Generates a catalog and suite under `integration/api_test_suites/<service>/`. It supports Swagger 2.0 and OpenAPI 3.x **JSON** (not YAML, not Postman). It maps fields, skips what it can't infer and picks an auth scheme.

The guided wizard (recommended)

```
/import-swagger --wizard ./openapi.json
```

1. **Swagger:** the spec URL or file, a suite name and the environment (no default; you pick).
2. **Endpoints:** every operation with its auth needs. DELETE and destructive writes start unselected.
3. **Test data:** Claude climbs a ladder and stops as soon as values are found:
   - the project (past run logs and reports, newest first, then your test data files),
   - read-only DB queries from `integration/*_db.json`,
   - the picked user's saved session (read only, no password),
   - the app's UI as the user you pick, reading real IDs and tokens from the app's own network calls. Captcha / OTP is handed to you.
4. **Cases:** a preview of 4 checks per endpoint: happy path (2xx + required fields), not found (404), no token (401), missing required field (400).
5. **Approve:** one approval writes the catalog and suite (never over an existing file) and opens the editor. **"Approve and run now"** also runs the suite.

> **WHERE THINGS LAND**
>
> Catalog and suite files are yours to commit. Collected values (`.agentex/cache/api-data/`) and captured tokens (`test/.auth/api-tokens/`) stay local and gitignored. Creating missing data through the UI needs your consent and an environment marked `"nonProduction": true` .

11 Database checks

↑ Back to contents

"The UI says success, but did the row really get written?" `db:` steps answer that by running only the named queries you defined, with values filled into their placeholders.

11.1 Step 1 — Define queries

integration/sample_db.json

```
{
  "name": "sample-db",
  "engine": "sqlserver",
  "connection": { "serverEnv": "DB_SERVER", "portEnv": "DB_PORT",
                  "databaseEnv": "DB_NAME", "userEnv": "DB_USER" },
  "queries": [
    { "name": "todo-by-title",
      "description": "Verify a todo row exists by title",
      "params": ["title"],
      "query": "SELECT TOP 1 Id, Title, Status FROM Todos WHERE Title = '{title}' ORDER BY Id
DESC" }
  ]
}
```

The connection values come from the active environment's `db` block. The password comes only from `SQLCMDPASSWORD` in `.env`.

11.2 Step 2 — Install sqlcmd

```
winget install -e --id Microsoft.Sqlcmd      # Windows (modern go-sqlcmd)
sqlcmd "-?"                                   # check it works
```

The current version targets SQL Server. On macOS / Linux, install Microsoft's sqlcmd for your platform.

11.3 Step 3 — Use it in a spec

```
3. db: sample-db.todo-by-title(title=qa-test-item) → expect 1 row
```

Behind the scenes Claude runs the bundled runner, which you can also run yourself:

```
node "<plugin-root>/skills/db-integration/scripts/run_db.js" \
  --entry sample-db.todo-by-title --param title=qa-test-item \
  --expect-rows 1 --log "$SESSION_DIR/logs/s6-todo-by-title.log"
```

Assertions: `--expect-rows <n>`, `--expect-min-rows <n>`. Output: one JSON line with PASS / FAIL / BLOCKED (exit codes 0 / 1 / 2).

11.4 Safety rules (enforced in code)

- **Catalog-only:** Claude never writes its own SQL.
- **DDL refused:** `DROP`, `TRUNCATE` and `ALTER` are blocked **even if catalogued**.
- **How parameters work:** `{title}` is **substituted as text** into the SQL, not bound as a database parameter. The runner allows only declared params, escapes quotes, and refuses values containing `;`, `--`, `/*` or a newline (BLOCKED). Keep catalogued queries read-only (`SELECT`) wherever possible.
- **Password** is never on a command line; sqlcmd reads `SQLCMDPASSWORD` itself.
- A bare `sqlcmd` command always prompts you (it's in the *ask* list).

12 UI check — compare with Figma or a screenshot

↑ Back to contents

A `ui-check:` step compares the live page, at that exact point in the scenario, with a design baseline.

12.1 Syntax

```
ui-check: figma <node-id | frame URL> — mode: exact|reference [ — viewport: desktop|tablet|
mobile|<W>x<H> ]
ui-check: image <path>               — mode: exact|reference [ — viewport: ... ]
```

12.2 Setup

Source	In the spec	One-time setup
`figma`	A node id (`123:456`) or the frame URL	`figma.fileKey` in `config/project.json` + `FIGMA_TOKEN` in `.env`
`image`	A path, e.g. `test/baselines/checkout-desktop.png`	None

```
"figma": { "fileKey": "AbCdEf123456", "token": { "envSecret": "FIGMA_TOKEN" } }
```

12.3 Modes

- **exact** — the page must match every visible detail. If a deviation might be rendering noise (fonts, anti-aliasing, dynamic dates), Claude asks you before deciding.
- **reference** — only the listed details decide the verdict. General layout drift passes **with a warning**.

EXAMPLE — REFERENCE MODE

```
ui-check: figma 123:456 — mode: reference
  - must: primary CTA reads "Pay now" and is enabled
  - must: order summary shows 3 line items with a total row
```

12.4 Viewports and results

- Defaults: desktop 1440×900, tablet 768×1024, mobile 390×844. Override with `"viewports": { "mobile": "414x896" }`.
- The form factor must match: a mobile baseline against a desktop run is a **View Mismatch**, not PASS / FAIL.
- Both images (`…-ui-check-baseline.png` and `…-ui-check-actual.png`) are saved. A failure becomes a normal defect with both images attached.

> **HONEST LIMIT**
>
> The verdict is a disciplined visual review, not a pixel diff. Very subtle deviations (1–2 px, near-identical colours) can be missed.

13 Ask the knowledge base

↑ Back to contents

Ask your project's knowledge base in plain language, standalone or mid-test. The answer is **advisory only**: it informs testing but never counts as PASS / FAIL proof.

13.1 Setup

```
"kb": { "baseUrl": "http://localhost:3000", "project": "acme-store" }   // config/project.json
KB_ASK_API_KEY=...                                                      # .env (if the server
needs it)
```

13.2 Use it

```
/ask-kb how does the checkout flow work?
/ask-kb acme-store: what fields are required at checkout?
```

Inside a spec:

```
## Scenarios
1. kb: how is a returning customer's discount applied at checkout?
2. **Verify** — apply the flow described above and confirm the discounted total in the UI.
```

Claude asks the KB first, then verifies the real behaviour in the browser. If the KB doesn't cover the question, it says so plainly. Claude never queries the KB on its own initiative.

14 Jira Cloud

↑ Back to contents

Estimate QA effort, design test artifacts from acceptance criteria and file bugs on Jira Cloud. Everything goes through bundled scripts over the Jira REST API v3 (no `acli`, no npm installs), always behind **one consolidated approval**.

> **BEFORE YOUR FIRST WRITE ON JIRA**
>
> As of v0.28.1, Jira **reads** were checked against a live site, but **creating** sub-tasks, test artifacts and bugs is covered by offline tests only. Try the first run on a test project, read the approval screen carefully, and check the ledger on the board afterwards.

14.1 One-time setup

1. **Run** `/init-test` and pick **Jira Cloud** at the tracker question (easiest). Or fill the block by hand:

   config/project.json

   ```
   "jira": {
     "site": "yourteam",          // → https://yourteam.atlassian.net (or a full URL)
     "project": "PROJ",           // the project KEY
     "board": "PROJ board",       // optional: steers sprint discovery
     "assignee": "qa@company.com" // optional default assignee(s)
   }
   ```

2. **Create an API token** at **id.atlassian.com → Security → API tokens → Create API token**.

3. **Put the credentials** in `.env`:

   ```
   JIRA_EMAIL=you@company.com
   JIRA_API_TOKEN=<paste the token>
   ```

4. **Done.** The `Bash(node:*)` permission already covers every Jira flow.

> **ONE TRACKER PER PROJECT**
>
> A config with both `azure` and `jira` blocks fails closed: "More than one tracker provider is configured". Keep exactly one.

14.2 What your Jira project needs

Feature	Needed for · how to enable
Sprints	`/estimate-sprint`. Team-managed: Project settings → Features → Sprints. Company-managed: use a Scrum board. Then **start** a sprint with stories.
Time tracking	Hours on `[Testing]` sub-tasks. The Time tracking field must be on the sub-task screen. If it isn't, sub-tasks are created without hours and the description says `Estimate: <n>h`; the approval screen warns you first.
Bug type	Bug filing. Add "Bug" to the project's work types.

14.3 Estimate QA effort

```
/estimate-sprint               # every story in the open sprint
/estimate-story PROJ-101 PROJ-102
```

1. **First run only:** with no `config/estimation.json` yet, the run reads the Original Estimates of `[Testing]` sub-tasks in your board's last **two closed sprints** and proposes the file (min / max per task). Abnormal values are listed with three choices: count it · use the suggested value · type a number. No history → labelled starter defaults.
2. **Rating.** Claude rates each story on the file's factors from 0 to 4. A script turns the ratings into hours inside each task's range. The same ratings always give the same hours.
3. **Questions.** Anything open (several open sprints, story already has tasks…) is asked in at most **one** question round.
4. **One screen, one approval.** You see the file, each story's ratings and hours, the sprint total and the exact write plan. Approve → `[Testing]` sub-tasks are created with time tracking and the label `testing`.

14.4 Design test artifacts — `/design-test`

```
/design-test PROJ-101
```

Jira has **no native Test Case type**. AgenTeX tells you this and asks what to create: your project's real issue types (sub-task types marked) or "document only". Steps are written as an ordered action → expected list in the description. Non-sub-task artifacts link to the story with a link type you choose (`Relates` recommended). Pin the choice in `.agentex/test-template.md` under `## Jira` so later runs skip the question.

14.5 File bugs after a run

1. **Ask:** *"File the defects from the last run as Jira bugs, parent story PROJ-101."*

2. **Validation.** Priority is checked against your project's real priority names. A severity custom field is used only if your Bug screen has one.
3. **Approve the plan.** Order: create Bug → attach screenshots (an annotated copy with a red box and label first) → link the story.
4. **Partial failure?** You get the created key, which attachments landed and the link state. Nothing is retried or cleaned up silently.

14.6 Known limitations on Jira

Gap	What AgenTeX does
No test plans / runs API	Test-plan operations refuse upfront. During bug filing you may link to an existing `/design-test` artifact instead.
No "Tested By" link	Uses a link type you choose (`Relates`).
Delete is permanent	Delete is never offered.
Live writes not yet proven	Reads were checked live; creates are covered by offline tests. Check the first run's ledger on the board.

14.7 Jira troubleshooting

Message	Fix
"Jira is not fully configured — missing: jira.site / jira.project"	Fill the `jira` block or re-run `/init-test` .
"Jira credentials are incomplete"	Add both `JIRA_EMAIL` and `JIRA_API_TOKEN` to `.env` .
401 / 403, or "answered anonymously with a 404"	The email must belong to the account that created the token. Regenerate the token.
"no Story sits in an open sprint"	Kanban board, no started sprint or empty sprint. Or estimate by key instead.
Unexpected "no duplicates found" / empty results	Jira may be answering anonymously. Check the credentials first.

15 Azure DevOps

↑ Back to contents

The same flows on Azure DevOps: estimation, test case design, bug filing, plus reading Azure resources mid-run. Board flows use bundled scripts over the ADO REST API; **no Azure CLI is needed** for them.

15.1 One-time setup

1. **Fill the `azure` block** (or pick Azure DevOps in `/init-test`):

   `config/project.json`

   ```
   "azure": {
     "org": "my-org",             // bare name or https://dev.azure.com/my-org
     "project": "MyProject",
     "team": "MyProject Team",
     "assignee": "qa@company.com"
   }
   ```

   Optional extras: `areaPath`, `iterationPath`, `bugTemplateId`, `testPlanId`, `valueArea`, `environment`, `bugCategory`, `apiVersion`.

2. **Create a Personal Access Token** in Azure DevOps (User settings → Personal access tokens). The repo does not list required scopes. A practical starting point is Work Items (read & write), plus Test Management if you mark test cases Failed. Give it the shortest expiry your team allows.

3. **Put it in `.env`:** `AZURE_PAT=<token>`. That's the whole setup. The scripts read it themselves and send it only in the Authorization header.

15.2 Estimate a sprint

```
/estimate-sprint                # every User Story in the current sprint
/estimate-story 12345 12346     # named stories
/estimate-sprint read the last 3 sprints    # first-run proposal from 3 sprints
```

Your team's method lives in `config/estimation.json` (commit it):

`config/estimation.json — the real starter defaults (used when there is no history)`

```
{
  "schemaVersion": 1,
  "tasks": [
    { "name": "Requirement Review",    "enabled": true, "minHours": 0.5, "maxHours": 1 },
    { "name": "Test Creation",         "enabled": true, "minHours": 0.5, "maxHours": 1 },
    { "name": "Test Execution",        "enabled": true, "minHours": 1,   "maxHours": 1.5 },
    { "name": "Bug Review and Retest", "enabled": true, "minHours": 0.5, "maxHours": 1 },
    { "name": "Automation",            "enabled": true, "minHours": 0.5, "maxHours": 0.5 }
  ],
  "factors": {
    "Scenarios / ACs": 1, "UI elements": 1, "Validations": 1, "Conditions / logic": 1,
    "Error messages": 1, "API integrations": 1, "Translations": 1, "Edge cases": 1
  },
  "source": { "kind": "starter-defaults" }
}
```

Totals: 3 h per story at every min, 5 h at every max. Task titles become `[Testing] Requirement Review` etc. Edit names, ranges and weights freely; unknown keys or `minHours > maxHours` stop the run with every problem named.

- **First run:** proposed from the Original Estimates of `[Testing]` tasks in the team's last two closed iterations (values over 3× or under ⅓ of normal are flagged).
- **Each story:** rated 0–4 per factor; hours placed inside each task's min–max range.
- **One approval** writes the file (first run) and one atomic create per task with its parent link, so an unparented task cannot exist.
- **Rebuild from history:** delete or move `config/estimation.json`.

15.3 Design test cases

```
/design-test 12345
```

1. **Analyse.** Claude reads the story's acceptance criteria and breaks them into test conditions, one test case per condition.
2. **Conventions.** Persona, journey steps, setup steps and languages come from `.agentex/test-template.md` (created on first run).
3. **Validate with zero writes.** It is a real User Story, duplicate titles are checked (a check that can't complete blocks), and steps are compiled to Steps XML.
4. **One screen:** conditions table, titled cases with step summaries, duplicate results and the write plan.
5. **Approve** → each case is created with its **Tested By** link, then a coverage check confirms every acceptance criterion is covered.

15.4 Approval and the ledger

All three flows end the same way: **one consolidated screen, one approval**. If something fails partway, you get an exact ledger: every intended write shown as done (ID + URL) or not done (reason). Nothing is retried or cleaned up without you.

15.5 File bugs after a run

1. **Ask:** *"File the defects from the last run as Azure bugs under story 12345."*
2. **Resolve.** Template, parent story, assignee and environment come from config and the run; at most one question round.
3. **Validate.** Parent story exists, no same-title duplicate, severity / priority / picklists match your project's real values, screenshots are checked and annotated.
4. **Review the screen.** It shows the recommended severity / priority with the reasoning, the evidence list and the write plan.
5. **Approve.** Order: upload attachments → create Bug → link parent story → set repro steps. Optionally mark the related test case **Failed**.

> **FIELD CACHE**
>
> Valid picklist values are cached in `.agentex/cache/tracker-fields-ado.json`. After an admin changes your process, ask Claude to *"refresh the tracker field cache"*.

15.6 Reach Azure resources mid-run

With the `az` CLI installed and logged in (`az login`), you can ask during a test:

```
"Check if the latest deployment succeeded."
"Tail the App Service logs while I run the checkout scenario."
"Does the export blob exist in the reports container?"
```

Read-only commands (`az webapp show / log`, `storage blob list`, `keyvault secret list`, `aks list`…) are pre-approved. Deploy, upload, `aks get-credentials` and `group create` always prompt. Secret values are never printed; Claude only confirms that they exist.

16 Reports and evidence

↑ Back to contents

Every run creates a new timestamped folder. You never create it by hand.

File	What it is
`report.md`	The consolidated summary: what passed, failed or was blocked, and why.
`extent-report.html`	A dark-themed dashboard: donut chart, status cards, expandable test cases with step details. All CSS / JS is inlined, so you can email it or attach it to a ticket.
`bugs/bug-list.md`	The merged defect list, ready for bug filing.
`browser-sessions/…` / `mobile-sessions/…`	Logs and one screenshot per scenario.
`api-tests/result.md`	Standalone API pass tally (`/test-endpoints`).
`run-summary.json`	The machine-readable run summary the dashboard is built from. Keep it.
`verdict.json`	CI runs only: the machine-readable verdict.

16.1 What a reported defect looks like

Every defect in `report.md` and `bugs/bug-list.md` follows the same format. An illustrative entry for the signup sample:

```
Title: Signup accepts a 3-character password without an error
Steps to reproduce:
  1. Open https://example.com/signup
  2. Fill Name "QA Tester", email qa.tester@example.com, password "abc"
  3. Click "Sign up"
Expected: A visible password length/strength error; the form does not submit
Actual:   The form submits and the success page is shown
Severity: High
Evidence: browser-sessions/<session>/screenshots/s4-weak-password.png;
          console: no errors; network: POST /api/signup → 201
```

FLAKY results (passed on retry) are listed separately under "Unstable results" and never go into `bug-list.md` .

16.2 The dashboard

The dashboard is generated for you at the end of every run. To regenerate it by hand, the script takes the run-summary JSON and an output path (see the plugin path note):

```
node "<plugin-root>/skills/extent-report/scripts/make_html_report.js" \
  executions/execu_<ts>/run-summary.json executions/execu_<ts>/extent-report.html
```

Run it from your **project root**; the input is the run's `run-summary.json` inside `executions/execu_<ts>/`. Easier: ask Claude *"regenerate the extent report for the last run"*.

> **COMMIT RUNS? CHECK THE SCREENSHOTS FIRST**
>
> `executions/` is **not** ignored by default, because many teams want evidence history in git. The secret scan skips screenshots, so review them for tokens or personal data. To keep runs local, add `executions/` to `.gitignore`.

17 CI quality gate

↑ Back to contents

Run a suite from your pipeline, headless, with no user interaction, and gate delivery on a reliable verdict.

17.1 Exit codes — the contract

Exit	Verdict	Meaning
0	PASS	Every scenario met expectations (an expected failure in a negative spec counts as a pass).
1	FAIL	**Real product defects** only, or warnings while `warningsFailGate` is on.
2	BLOCKED	**Environment / infrastructure**: unreachable target, missing secret, timeout, captcha, a session that never concluded.

> **THE PROMISE**
> An environment failure never produces exit 1, and a real defect is never hidden behind an environment excuse.

17.2 The command

```
node "<plugin-root>/skills/test-execution/scripts/ci_gate.js" --suite test/suite1/ --env uat

# run from the AgenTeX project root. scope: exactly one of
--spec <file> [--spec <file> ...]  |  --suite <folder>  |  --all
# policy overrides (flags > config > defaults)
--retries 3  --timeout-minutes 60  --warnings-fail true  --flaky-fails-gate false
```

stdout carries exactly **one JSON line** (the verdict); diagnostics go to stderr. Most BLOCKED results are retried automatically (up to 3 times by default, 30 s apart, each in a fresh session); PASS / FAIL are never re-rolled.

BLOCKED reason	Retried?
Target unreachable, timeout, missing secret, `no-verdict` with unknown cause…	Yes
`session-error` / `not-authenticated` (no Claude credential)	**No** — set `ANTHROPIC_API_KEY`
`no-verdict` / `tool-denied` (CI settings deny a tool)	**No** — add the allow rule to `ci-settings.json`
`preflight-driver` (a `Drivers: mobile` spec in scope)	**No** — remove mobile specs from the CI scope
`unstable` (flaky, with `flakyFailsGate` on)	**No** — by design

17.3 Runner prerequisites

1. **Node 20+** and the Claude CLI: `npm install -g @anthropic-ai/claude-code`, with `ANTHROPIC_API_KEY` (or Bedrock / Vertex credentials) from your secret store.
2. **The AgenTeX plugin** installed, or a pinned checkout of the repo as `<plugin-root>`.
3. **The browser driver** in the project: `npm install -D @playwright/cli && npx playwright-cli install-browser chromium`.
4. **Every `envSecret` name** injected as an env var from the secret store. A CI runner has no `.env`.

17.4 Example — GitHub Actions

Copy the shipped template `skills/test-execution/templates/ci/github-actions.yml` into `.github/workflows/agentex-gate.yml` and replace every `<placeholder>`. The job's steps, in order:

```
- uses: actions/checkout@v4
- uses: actions/setup-node@v4
  with: { node-version: 20 }
- run: npm install -g @anthropic-ai/claude-code
- run: claude plugin install <agentex@your-marketplace>
- run: npm install -D @playwright/cli && npx playwright-cli install-browser chromium
- name: AgenTeX gate
  # continue-on-error: true      # ADVISORY mode: a PM approval becomes the gate
  env:
    ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
    QA_PASSWORD:       ${{ secrets.QA_PASSWORD }}
    API_TOKEN:         ${{ secrets.API_TOKEN }}
  run: node "<plugin-root>/skills/test-execution/scripts/ci_gate.js" --suite test/suite1/ --
env uat
```

The template then writes the verdict to the job summary and uploads `executions/execu_*/` as the `agentex-run` artifact. A second job waits for a PM approval: create an environment named `agentex-pm-approval` in **repo Settings**

→ **Environments** and add **Required reviewers**, otherwise it passes without anyone approving. Set the job `timeout-minutes` above `(1 + retries) × timeoutMinutes`.

For Azure Pipelines use `templates/ci/azure-pipelines.yml` (with a `ManualValidation` PM approval step). Publish `executions/execu_*/` as the artifact. **Never publish `test/.auth/`.**

17.5 CI mode rules

- **No questions:** anything that needs a human becomes BLOCKED with the exact question in `blockedReasons`.
- **No tracker writes:** bug filing stays interactive; file the bugs from `bugs/bug-list.md` later.
- **Passwords** declared as `envSecret` are typed by `fill_secret.js`; the session never sees the value. Missing → `secret-unavailable`.
- **Mobile specs** are not run in CI by the browser gate (`preflight-driver`).
- Team policy can live in `config/project.json`: `"ci": { "warningsFailGate": true, "retries": 3, "timeoutMinutes": 60, "flakyFailsGate": false }`.

18 Git and security

↑ Back to contents

18.1 What to commit

Commit	Keep local (gitignored)
`config/project.json`, `config/estimation.json`, `environments/*.json` (**after review**: no real customer data, phone numbers or plain-text passwords), `integration/`, `test/` specs, `test/baselines/`, `CLAUDE.md`, `.claude/settings.json`, `.agentex/version.json`, `.agentex/test-template.md`	`.env`, `test/.auth/` (saved sessions and API tokens), `.playwright-cli/`, `test/.ui-baselines/`, `.agentex/cache/`, `.agentex/scratch/`

18.2 Environment safety

Environment	Recommended use	Restrictions
QC / dev	Everything: sequential, parallel, define-flow, API writes, Swagger wizard.	Can be marked `"nonProduction": true`.
UAT / staging	Regression, CI gate, read-mostly API / DB checks.	Review catalogued write requests first; use shared QA accounts only.
Production	Not recommended. At most read-only smoke checks, agreed with the owner.	No write requests in the catalog, no `/define-flow`, never `nonProduction`, no production credentials on test machines.

18.3 Secret scan and pre-commit hook

- The plugin **never sets a remote and never pushes**. Add your remote yourself with a URL that carries no credentials: `git remote add origin https://git.example.com/team/qa-project.git`.
- `/init-test` offers an optional **pre-commit hook** that runs the secret scan and refuses a commit that leaks a secret.
- **Screenshots are not scanned.** The secret scan skips binary files, so a screenshot that shows a token, a customer's data or an OTP can reach git if you commit `executions/`. Review evidence before committing it, or add `executions/` to `.gitignore`.
- **Narrow** `Bash(node:*)` on shared or production-adjacent machines. It allows *any* node command, not just AgenTeX's scripts (see 3.5).
- **Test against non-production environments.** Catalogued API writes and `/define-flow` steps you approve really run. Only mark an environment `"nonProduction": true` if it truly is.

- Findings include `secret-value`, `env-file-tracked`, `remote-credential` (leaks) and `plain-secret-field` (warning). Fix them by moving the value into `.env` and referencing it with `envSecret`.

19 Keeping your project up to date

↑ Back to contents

1. **Commit or stash** your changes. `/update-agentex` requires a clean git tree, because git is the rollback.
2. **Run** `/update-agentex`. It first checks for a newer plugin version and asks once before pulling. After a pull, reload Claude Code and run it again.
3. **Migration.** Folder structure, config schemas, the secrets-only `.env` split, the catalog folder, `.gitignore` and `CLAUDE.md` are updated. **Your values are carried over, never reset.**
4. **Review the report.** Specs, old runs and filled-in catalogs are never rewritten; any convention drift in them is only flagged.

Safe to re-run: an up-to-date project reports "already up to date" with zero writes. `.agentex/version.json` records the version the project matches.

20 Troubleshooting and FAQ

↑ Back to contents

20.1 Common problems

Problem	What to do
Claude asks permission before every browser command	Copy the `permissions` block into `.claude/settings.json` (3.5).
A step is BLOCKED "not in catalog"	Add the request / query to `integration/*_api.json` or `*_db.json` first; names must match exactly.
A step is BLOCKED: user not defined	Add the user handle under `users` in the active `environments/<env>.json`.
"Environment uat not found"	Create `environments/uat.json`. Missing environments are never silently replaced.
A mobile spec is BLOCKED in `/execute-test`	Correct behaviour: run it with `/execute-mobile-test`.
`webdriverio not found`	`npm install -D webdriverio` in your project.
Appium session fails	Is the server running (`npx appium server`)? Is the device in `adb devices`? Is the driver installed (`npx appium driver list`)?
Saved login no longer works	Nothing to do: the run logs in again and saves a fresh session. Apps using IndexedDB cannot be resumed.
Captcha / OTP during a run	Complete it in the visible browser window. In CI it is BLOCKED `captcha-or-otp`.
Swagger file is YAML	Convert it to JSON, or use the JSON endpoint (e.g. `/v3/api-docs`, `/swagger.json`).
Tracker rejects a field value	Ask Claude to "refresh the tracker field cache".
CI exit 2 with `tool-denied`	Add a matching allow rule to your `ci-settings.json`.
CI exit 2 with `not-authenticated`	Set `ANTHROPIC_API_KEY` on the gate step.

20.2 Supported, not supported, experimental (v0.28.1)

Area	Status	Notes
Browser (Chromium via playwright-cli)	Supported	Sequential, parallel, CI.
Android / iOS native apps	Supported	Not run by the CI gate; iOS needs macOS.
API: OpenAPI 3.x / Swagger 2.0 JSON	Supported	YAML and Postman collections not supported.
Database: SQL Server	Supported	Other engines (PostgreSQL, MySQL) not yet.
Azure DevOps boards	Supported	Estimation, test design, bugs, test-case Failed.
Jira Cloud	**Writes experimental**	No test plans / runs, no Tested-By link, no delete; Jira Server / Data Center not covered.
UI check	Supported	Visual judgement, not pixel diff.
Session reuse	Supported	Not for IndexedDB-based logins.
Captcha / unobtainable OTP	Not supported (by design)	Human step interactively, BLOCKED in CI.

20.3 Frequently asked questions

Does AgenTeX change my application? Not by design: test runs write to `executions/`, and setup commands write AgenTeX's own project files (1.2 lists them). The guarantee is enforced only when the `Edit(...)` deny rule points at your real source folder.

Can it see my passwords? Not if you copied the permission rules: they deny reading `.env`. The bundled scripts read values themselves and send them only in headers or type them via `fill_secret.js`.

Sequential or parallel? Sequential for new or exploratory testing, where you want to watch and steer. Parallel for regression suites you trust.

Can one spec mix UI, API and DB? Yes. A browser spec can contain `api:`, `db:`, `kb:` and `ui-check:` lines.

Will it write to Jira / Azure without asking? Never. Every tracker flow ends at one consolidated approval screen, and CI mode blocks all tracker writes.

B Appendix — Glossary

↑ Back to contents

Term	Meaning
Spec	A markdown file describing one feature or flow to test: target, acceptance criteria, scenarios.
Suite	A folder of specs run together, e.g. `test/suite1/`.
Scenario	One numbered test inside a spec (happy path, negative case…).
Stateful chain	Scenarios that depend on each other and must run in order in one session.
Driver	What executes a step: browser, api, db, kb, ui-check, mobile.
Catalog	`integration/*_api.json` / `*_db.json`: the only API calls and queries tests may run.
Suite file (API)	`*_suite.json`: concrete params and expected results for catalog entries.
Environment	`environments/<env>.json`: URL, users and connections for qc / uat / live.
envSecret	A reference by name to a secret in `.env`; the JSON never holds the value.
Sequential / Parallel	Human-in-the-loop run with pauses / autonomous run with one sub-agent per spec file.
Sub-agent	An independent Claude worker: `qa-executor`, `mobile-qa-executor`, `api-executor`.
Evidence	Screenshots and logs saved for every scenario, pass or fail.
BLOCKED	Could not be judged because something is missing or the environment failed. Not a product defect.
Consolidated approval	The single screen showing every tracker write before anything is written.
Ledger	The exact list of tracker writes done / not done after a partial failure.
Verdict	The CI result: PASS (0), FAIL (1) or BLOCKED (2), plus `verdict.json`.
<plugin-root>	The folder where AgenTeX is installed (3.8).

C Appendix — Cheat sheet

↑ Back to contents

```
# ---- install (once) ----
/plugin marketplace add MhmdElGazzar/elgazzar-plugins
/plugin install agentex@elgazzar-plugins
npm install -D @playwright/cli && npx playwright-cli install-browser chromium
/init-test

# ---- browser ----
/execute-test https://example.com
/execute-test suite2 on uat
Run a parallel regression against https://example.com from the specs in test/suite1/.
/define-flow https://example.com on uat
/define-flow test/suite1/checkout.md

# ---- mobile ----
npm install -D appium && npx appium driver install uiautomator2
npx appium server
/execute-mobile-test suite1

# ---- API / DB ----
/test-endpoints
/test-endpoints --edit
/import-swagger ./openapi.json --name acme-api
/import-swagger --wizard ./openapi.json
api: sample-api.get-todo(id=1) → expect HTTP 200 and title present
db:  sample-db.todo-by-title(title=qa-test-item) → expect 1 row

# ---- design / KB ----
ui-check: figma 123:456 — mode: exact — viewport: desktop
ui-check: image test/baselines/checkout-desktop.png — mode: reference
/ask-kb acme-store: how does the checkout flow work?

# ---- Jira / Azure DevOps ----
/estimate-sprint
/estimate-story 12345 12346          # or PROJ-101 PROJ-102 on Jira
/design-test 12345
"File the defects from the last run as bugs under story 12345."

# ---- CI / maintenance ----
node "<plugin-root>/skills/test-execution/scripts/ci_gate.js" --suite test/suite1/ --env uat
/update-agentex
```

Based on AgenTeX v0.28.1 — github.com/MhmdElGazzar/agentex (MIT). Created by Mohamed Elgazzar and contributors.